

Práctica 1: Conceptos y mecanismos básicos

Objetivos y requisitos

Objetivos

Los objetivos que se plantean en esta práctica son :

- Analizar latencias de protocolos básicos de red.
- Diseñar e implementar un algoritmo básico de sincronización de barrera distribuida.
- Familiarización con la arquitectura cliente-servidor y master-worker.

Requisitos

- Golang versión ≥ 1.22 linux/ARM64/amd64¹
- Seguir, de forma rigurosa, la metodología de programación estructurada y organizada, con nombres significativos (tipos, variables, parámetros, funciones), y con funciones cortas (no más de 12 líneas de instrucciones). No más de 80 caracteres por línea.
- El protocolo *ssh* para ejecutar comandos remotos
- Los ficheros de apoyo, que acompañan al guión, con un esqueleto de código fuente en el que inspirarse y apoyarse.

Especificación Técnica de las Máquinas del Lab1.02

Para las prácticas de esta asignatura utilizaremos el cluster de Raspberry Pi 4 Model B del Laboratorio 1.02. Para pruebas intermedias, podéis utilizar también las 20 máquinas de sobremesa del laboratorio L1.02, con arquitectura hardware AMD64.

El procesador de una Raspberry Pi 4 Model B (ARM Cortex-A72) tiene 4 cores con 1 hilo por core, que nos van a permitir ejecutar, por lo tanto, hasta 4 instancias en

¹Disponible en <https://golang.org/doc/install>

Práctica 1: Conceptos y mecanismos básicos

paralelo de nuestra aplicación (la especificación técnica de la CPU podéis obtener de forma detallada si ejecutáis la instrucción `lscpu` en el sistema operativo Linux). Estas máquinas cuentan con 8 GB.

1. Ejercicios

En esta práctica tenéis que realizar los ejercicios siguientes (*no necesariamente en este orden*):

Ejercicio 1.**Obligatorio.**

Análisis de las prestaciones de red descritas en la Sección 2.1:

- Ejecutar repetidamente los escenarios descritos en la Sección 2.1, recoger los tiempos derivados de la ejecución en todos los escenarios y crear una tabla para la memoria de esta práctica.
- Explicar a qué actividades se debe el coste de ejecutar la operación Dial de TCP en Golang y estudiar la variación en el tiempo de ejecución, cuando Dial falla, cuando funciona correctamente y cuando el cliente y el servidor están en la misma máquina y en distinta máquina.
- Explicar a qué actividades se debe el coste de ejecutar la transmisión en el escenario de UDP.

Ejercicio 2.**Obligatorio.**

Diseñar e implementar un algoritmo básico de sincronización de barrera distribuida, con ayuda del código suministrado en el directorio "barrier".

Práctica 1: Conceptos y mecanismos básicos

Ejercicio 3.**Obligatorio.**

Diseñar (diagramas de secuencia) e Implementar (en Go) las 2 arquitecturas software descritas en la Sección 2.3, para la ejecución de una aplicación distribuida de cálculo de número primos :

- cliente servidor concurrente con un pool fijo de Goroutines
- máster-worker. En este caso, el máster, mediante comandos ssh, arrancará los workers antes de operar. De manera que no los tendréis que arrancar vosotros manualmente. El máster obtendrá de un fichero un listado de máquinas (IPs + puertos) donde puede lanzar workers.

En ambas hay que utilizar el paquete gob para serializar los mensajes.

Entrega: Diagramas y el código de los servidores, del máster y del worker

Ejercicio 4.**Obligatorio.**

Escribir una memoria donde se muestren todos los resultados y que obligatoriamente tendrá estas secciones:

- Sección de Introducción o Descripción del Problema, descripción de la aplicación y de los recursos computacionales disponibles.
- Sección de Diseño de las 4 arquitecturas (Maquinas de estado, diagramas de secuencia, si fuera necesario añadir diagrama de componentes).

Entrega: memoria en pdf

1.1. Instrucciones de Entrega y Defensa

Deberéis entregar un fichero zip que contenga: (i) El código fuente modificado y creado durante la práctica, y (ii) la memoria en pdf. La entrega se realizará a través de moodle en la actividad habilitada a tal efecto. **La fecha de entrega será no más tarde del día anterior al comienzo de la siguiente práctica.**

Durante la sesión de la práctica 2 se realizará la revisión de la práctica 1 con pruebas de ejecución distribuida en el cluster de raspberry pi.

2. Descripción de los ejercicios a realizar

2.1. Estudio de la Red de Comunicación

Para estudiar las características de la red de comunicación que tienen las Raspberry Pis vamos a realizar distintos experimentos que debéis reflejar en la memoria. En los escenarios vamos a considerar los protocolos TCP y UDP, por tanto, para poder explicar las mediciones que obtengamos, tendremos que comprender cómo se comportan los distintos protocolos.

Se proporciona el código de un cliente TCP (en el directorio network), que primero se conecta a un servidor que no está en marcha y luego lanzaremos el servidor para que interactúen. Se trata de medir el tiempo que tarda en devolver el error la llamada "Dial" en el primer caso y el tiempo que tarda "Dial" en el segundo caso. Comprobar si se obtienen los mismos valores cuando el cliente y el servidor están en la misma o en distinta máquina.

El segundo escenario se corresponde a un cliente y servidor UDP sencillos, se trata de medir el tiempo que le cuesta al cliente el envío de una letra y la recepción de otra letra. Comprobar si se obtienen los mismos valores cuando el cliente y el servidor están en la misma o en distinta máquina.

2.2. Modelo de Aplicación de uso intensivo de CPU

El modelo de aplicación que utilizaremos en esta práctica es uno de los más sencillos posibles desde un punto de vista de los sistemas distribuidos. La aplicación consiste en una única tarea que recibe la entrada, proporciona una salida y termina. En particular, la aplicación de esta práctica consiste en encontrar los números primos dentro de un intervalo [1000, 70000] dado como argumento. Es importante resaltar que al haber una única tarea, no hay interdependencias con otras tareas. Por eso, es uno de los tipos de aplicación más sencillos desde un punto de vista de los sistemas distribuidos. La aplicación es sobre todo intensiva en CPU, puede no requiere de mucha red de comunicación y no requiere de almacenamiento.

Junto con este enunciado, tenéis disponible unos ficheros de ayuda:

- Un cliente ya terminado y que no hay que cambiar que envía peticiones / tareas con un intervalo que siempre es el mismo [1000, 70000]. Simplemente lo invocaremos con distintos parámetros para que genere cargas de trabajo diferentes.
- Un servidor, por completar, que contiene una función para resolver esta práctica. En particular: IsPrime y FindPrimes. La primera determina si un número entero dado es primo o no y devuelve true o false, respectivamente. La segunda toma como entrada un intervalo y devuelve un array con todos los números primos en el intervalo.

2.3. Arquitecturas Cliente-Servidor y Máster Worker

La arquitectura cliente servidor es una de las más utilizadas en sistemas distribuidos. En esencia, consiste en un proceso servidor, que aglutina la mayor parte de la funcionalidad, y un conjunto de procesos clientes que solicitan al servidor esa funcionalidad mediante el intercambio de mensajes. Existen distintas variantes de esta arquitectura, así como distintas posibilidades de implementación que nos proporciona Go:

1. **La arquitectura cliente servidor secuencial** consiste en un servidor que atiende peticiones de forma secuencial, de una en una, de manera que cuando llegan varias peticiones, atiende una de ellas (a menudo la primera en llegar) y, una vez terminada, atiende la siguiente. Para reducir el tiempo de espera de los clientes, siempre que haya recursos hardware suficientes en el servidor y *siempre que la aplicación lo permita*, se puede utilizar la arquitectura cliente-servidor concurrente.
2. **La arquitectura cliente servidor concurrente** consiste en un servidor que puede atender varias peticiones en paralelo. En Go esto puede implementarse de varias formas:
 - La más sencilla consiste en que inicialmente el servidor espera a que llegue una petición, una vez recibida se crea una Goroutine y se le pasa la petición para que la procese y devuelva el resultado.
 - Crear una Goroutine por cada petición conlleva un sobrecoste en tiempo y, por tanto, otra opción podría ser tener un conjunto fijo de Goroutines (Goroutine pool en inglés, patrón software pool de elementos) que atienden peticiones y se pueden reutilizar. Las Goroutines se comunican con el proceso principal servidor a través de dos canales síncronos: un canal donde el programa principal envía las tareas que se reciben y las Goroutines leen todas de ese canal y van extrayendo los datos de él (se realiza de forma oportunista, esto es, la primera que consigue obtener la tarea se la lleva) y otro canal donde las Goroutines escriben los resultados para que el programa principal del servidor los recoja. Alternativamente, podríamos prescindir de este canal de resultados y que las Goroutines enviaran directamente el resultado al cliente correspondiente. Podéis encontrar información al respecto y código fuente de ejemplo en Go en este link ².
 - La tercera y última opción consiste en utilizar la función `Select` del paquete `socket`, que es equivalente a la función `Select` del lenguaje C. Esta funcionalidad tiene su origen en el SO UNIX. Hay llamadas especializadas en algunos sistemas operativos, como "epoll" en Linux o "kqueue" en FreeBSD, que optimizan aún más el multiplexado en accesos a descriptors de sockets y ficheros. Los diferentes elementos que operan sobre la técnica "epoll" de Linux están también disponibles en el package "syscall" de Go. Y estas son técnicas, no solo disponible en su librería estándar, sino utilizadas, directamente, en el runtime de Go.

²<https://golangbot.com/buffered-channels-worker-pools/>

Práctica 1: Conceptos y mecanismos básicos

3. **La Arquitectura Máster Worker** puede verse como una extensión de la arquitectura cliente-servidor concurrente con un pool de Goroutines, en la que existe un programa principal (máster) que reparte las tareas a un conjunto de procesos (workers). Sin embargo, a diferencia del cliente servidor, en el master worker, cada Goroutine no hace uso de los recursos propios de la máquina sino que interactúa con otro proceso remoto en otra máquina. Esta característica nos proporciona la ventaja de posibilitar la escalabilidad.

3. Compilación, Ejecución y Depuración en Golang

3.1. Paquetes, modulos, compilación, ejecución

En las máquinas del Lab 1.2 está instalado el compilador de Golang v1.22 y v1.24. Esta versión del compilador será la referencia durante la asignatura. Para utilizarlo en las máquinas del laboratorio, podéis obtener la ruta en la que está instalado:

```
lab102-198:~/ which go
/usr/local/go/bin/go
```

Y añadirlo en la variable de entorno *PATH*:

```
lab102-198:~/ export PATH=$PATH:/usr/local/go/bin
```

Se puede incluir esa sentencia en el fichero *.bashrc* de manera que el intérprete de comandos la ejecute siempre al acceder a vuestra cuenta.

La unidad básica de organización y acceso de código es el "paquete" (package), también para nuevos programas cuando su tamaño crece y se desea estructurar. Cada paquete se ubica en su propio directorio cuyo nombre es idéntico al paquete. Para simplificar el acceso y utilización de paquetes locales a un programa, es aconsejable organizarlos dentro de un "modulo" Golang como elemento organizativo que agrega paquetes y, si es necesario crear programas ejecutables, varios paquetes main. El acceso del código main a los paquetes internos al modulo requieren explicitar un camino local de importación de dichos paquetes con el formato :

```
import "<nombre modulo>/<path de acceso hasta directorio de paquete>".
```

El código esqueleto que acompaña este guión se puede ver como un ejemplo simple de esta metodología³.

Dicho código esqueleto está ubicado en un modulo llamado "practical". El fichero "go.mod" que define este modulo lo teneis ya generado mediante la ejecución del comando "go mod init practical", una vez ubicados en el directorio "practical". Ahora los paquetes y códigos main internos utilizaran PAtHs locales a dicho modulo. Para compilar y ejecutar el programa principal de cliente en el esqueleto suministrado, primero os situais en el directorio "practical" y ejecutais :

³Otro ejemplo significativo de uso de modulo lo podeis ver en <https://www.digitalocean.com/community/tutorials/how-to-use-go-modules>

Práctica 1: Conceptos y mecanismos básicos

```
lab102-198:~/ go build cmd/client/main.go
lab102-198:~/ ./client
```

o tambien

```
lab102-198:~/ go run cmd/client/main.go
```

La compilación de ambos ejecutables, servidor y cliente, con destino al directorio relativo bin, lo podeis realizar con el comando :

```
lab102-198:~/ go build -o bin ./...
```

El acceso a la librería "com" puesta a disposición desde fuera de dicho paquete, pero dentro del modulo "practical" se efectua mediante :

```
import "practical/com"
```

Finalmente, para reejecución automática de un comando shell (por ejemplo, "go run ./cmd/server/main.go") tras el evento de guardar un fichero de código ".go", eventualmente modificado, podeis utilizar, entre otros, la herramienta "arelo" que podeis encontrar en <https://github.com/makiuchi-d/arelo>. Teneis un par de ficheros de ejemplo, que podeis utilizar, en el código esqueleto asociado a este guión. Para que funcione, debe estar instalado *inotify* en Linux.

3.2. Depuración

Golang dispone de depuradores de estilo tradicional como Delve⁴. Se puede integrar a VScode.

Pero para depuración de programas distribuidos es más adecuado obtener trazas con estampillas de tiempo, mediante sistemas de registro de eventos o "logging", para poder obtener un orden de ejecución de las partes distribuidas, y en particular poder analizar problemas de paso de mensajes y sincronización.

Los mensajes mostrados permiten realizar un análisis de una traza de ejecución una vez el programa se ha ejecutado. Una forma rudimentaria de registrar eventos consisten en escribir mensajes por pantalla (bien salida estándar o bien en la salida de error), haciendo uso de las bibliotecas de entrada salida.

Sin embargo, los lenguajes de programación modernos incorporan librerías específicas para registrar eventos (tiempo, fichero, línea de fichero). En Golang, está el paquete "log", que puede ser útil para depurar programas. Podeis ver su API aquí:

<https://pkg.go.dev/log>

⁴Documentación de Delve <https://github.com/go-delve/delve/tree/master/Documentation>

Práctica 1: Conceptos y mecanismos básicos

Especial interes tienen la función "SetFlags" (precisar microsegundos, fichero y nº de línea) y las diferentes funciones "print" del paquete "log" de librería estándar. Por ejemplo, para ello, la configuración del paquete log sería la siguiente:

```
// configurar para que logs salgan con estampillas en
// microsegundos, nombre de fichero y línea de los prints
log.SetFlags(log.Lshortfile | log.Lmicroseconds)
```

Luego en el código:

```
log.Println("Client send Request with Id and Interval : ", request)
```

Para evitar problemas de precisión entre los relojes físicos sincronizados por NTP, se pueden introducir pequeños retardos en la función "send" (Time.sleep(1 * Time.Millisecond)) para asegurar el umbral de relevancia de relojes físicos para ordenación de eventos.

4. El Despliegue del Software en un Sistema Distribuido

El desarrollo y pruebas iniciales de vuestra solución se puede realizar de forma local en un portátil u ordenador de sobremesa. Pero las pruebas y evaluación finales **deberá** ser realizado, en todas las prácticas de la asignatura, en el cluster de 20 máquinas (raspberris-ARM64-), con Ubuntu server 22.04, que se encuentra en el laboratorio 1.02, y que nos permiten un entorno dedicado y controlado de ejecución distribuida.

Cada pareja de prácticas tiene un rango exclusivo de 10 puertos TCP/UDP, *asignado en un documento que tenéis disponible en moodle*, para no interferir entre parejas. Para distribuir el uso de las 20 máquinas del cluster, cada pareja puede utilizar hasta 4 máquinas, cuyo rango está también definido en el mismo documento en moodle. El nº de máquina en el rango se utiliza como última cifra de su dirección IP (máquina 7, IP 192.168.3.7).

Estas máquinas son accesibles remotamente, con direcciones IP privadas en el *rango 192.168.3.1-20*, sólo por ssh desde el servidor *central.cps.unizar.es*, utilizando vuestras cuentas de hendrix (mismo nombre y contraseña). Vuestras cuentas de usuario tienen un *home* local con una cuota máxima de 300 MB. Configurar en ellas la autenticación ssh/scp mediante clave pública (authorized_keys). **No** utilizar el servidor central para ejecutar vuestros programas, utilizarlo, sólo, para copias de ficheros y pasarela de acceso ssh al cluster de 20 máquinas.

Cada alumno dispone de su propio *directorio /misc/alumnos/sd/sd2526/aXXXXXX* (mediante el Sistemas de Ficheros Distribuido NFS), *compartido* entre las 41 máquinas de laboratorio 1.02 (20 de sobremesa, 20 ARMs del cluster y servidor central), para copiar en él el código :

```
$ scp -r practical central.cps.unizar.es:/misc/alumnos/sd/sd2526/aXXXXXX
```

y compilarlo para cada plataforma, AMD64 o ARM64. Cuando vayamos a ejecutarlo en varias máquinas, podremos acceder a él desde cualquier punto, a través del NFS (basta con entrar en vuestra cuenta en todas las máquinas donde queramos ejecutar el código).

Se puede, también, realizar una compilación cruzada en una máquina AMD64 para generar binario ARM64 mediante :

Práctica 1: Conceptos y mecanismos básicos

```
$ GOOS=linux GOARCH=arm64 go build main.go
```

Para *pruebas intermedias*, podéis utilizar también las 20 máquinas AMD64 de sobremesa del laboratorio L1.02.

4.1. La Ejecución Remota de Scripts en Unix: SSH

Secure Shell (SSH) es, por un lado, un protocolo de red que permite acceder a un servidor de forma remota y segura, ejecutar comandos remotos y copiar ficheros (scp). Una aplicación muy habitual de SSH es proporcionar un acceso a la línea de comandos del servidor de forma remota, pero también permite ejecutar comandos shell de forma remota.

Por ejemplo, si se ejecuta el comando SSH de OpenSSH como en el siguiente fragmento de código (server1 puede ser la IP o el nombre DNS):

```
$ > ssh user1@server1 command1
```

se consigue que el usuario user1 ejecute el comando command1 de forma remota en el servidor server1. Por defecto, ssh solicitará al usuario user1 que introduzca su password.

Le acompaña otro comando de copia, "scp", que utiliza la infraestructura y opciones del comando de ssh para copiar fichero entre máquinas remotas (incluida la opción -r para copias recursivas). Ejemplo :

```
$ > scp -r * user1@server1:/<path directorio remoto>
```

4.2. Ejecución Remota de Scripts sin Password

Es posible iniciar sesión en un servidor Linux remoto sin tener que introducir interactivamente la contraseña, utilizando la autenticación con clave pública. Esto permite automatizar la ejecución de determinados scripts.

Vamos a asumir que estamos trabajando en una máquina cuya IP es 155.210.154.200 (local-host) y que queremos conectarnos remotamente a la máquina 155.210.154.201.

Una forma de configurarlo es mediante los siguientes 3 pasos, usando ssky-keygen y ssh-copy-id.

Paso 1: Desde la línea de comandos de 155.210.154.200 vamos a crear las claves pública y privada mediante ssh-key-gen :

```
rafaelt@local-host$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/home/rafaelt/.ssh/id_rsa): [Enter key]
Enter passphrase (empty for no passphrase): [Press enter key]
Enter same passphrase again: [Press enter key]
Your identification has been saved in /home/rafaelt/.ssh/id_rsa.
Your public key has been saved in /home/rafaelt/.ssh/id_rsa.pub.
The key fingerprint is:
33:b3:fe:af:95:95:18:11:31:d5:de:96:2f:f2:35:f9 rafaelt@local-host
```

Práctica 1: Conceptos y mecanismos básicos

Paso 2: Copiamos la clave pública en nuestra cuenta del servidor remoto (155.210.154.201) utilizando `ssh-copy-id`:

```
rafaelt@local-host$ ssh-copy-id -i ~/.ssh/id_rsa.pub 155.210.154.201
rafaelt@155.210.154.201's password:
Now try logging into the machine, with "ssh 'remote-host'", and check in:
.ssh/authorized_keys
to make sure we haven't added extra keys that you weren't expecting.
```

Paso 3: Ahora ya se pueden utilizar `ssh` sin necesidad de introducir el password de forma interactiva :

```
rafaelt@local-host$ ssh 155.210.154.201 command1
Last login: Sun Nov 16 17:22:33 2008 from 192.168.1.2
```

De manera que ejecutará el comando `command1` en el servidor 155.210.154.201.

4.3. SSH desde Golang

Go cuenta con una librería que implementa la funcionalidad de SSH, lo cual posibilita la interacción con servidores SSH directamente dentro de un programa desde código Go, en lugar de utilizar invocaciones externas. Por ahora no lo utilizaremos, pero puede tener interés si se necesita mayor integración de funcionalidades.

5. La Concurrency en Go

La concurrencia es una parte inherente del lenguaje de programación Go y para ello el lenguaje proporciona dos elementos fundamentales: las Goroutines y los canales síncronos. Las Goroutines son funciones o métodos que se ejecutan al mismo tiempo que otras funciones o métodos. Los goroutines pueden considerarse hilos (*threads*), pero en realidad la sobrecarga (*overhead* en inglés) asociada a una Goroutine es mínima en comparación con la de un thread. Por lo tanto, es común que las aplicaciones Go tengan miles de Goroutines ejecutándose al mismo tiempo, de hecho, están diseñadas para tal fin. Las Goroutines se multiplexan en una menor cantidad de threads. Incluso, podría darse el caso en que solo hubiera un thread en un programa con miles de Goroutines.

En el Fragmento de Código Gorutina simple ⁵, puede observarse el mecanismo, muy simple, con el que se diseñaron las Goroutines. En el código existen dos funciones `hello()` y `main()`, el programa principal. Una vez que se ejecuta `main`, en la línea 11 se ejecuta `go hello`, lo que hace que se cree una Goroutine que ejecuta la función `hello` simultáneamente con el programa principal `main`. Muy probablemente, el programa principal terminará antes que la Goroutine, finalizando la ejecución de todo el programa, de manera que a la Goroutine no le dará tiempo a ejecutarse completamente.

⁵Disponible y ejecutable en https://play.golang.org/p/zC78_fc1Hn

Práctica 1: Conceptos y mecanismos básicos

```

1 package main // Código Gorutina simple
2
3 import (
4     "fmt"
5 )
6
7 func hello() {
8     fmt.Println("Hello world goroutine")
9 }
10 func main() {
11     go hello()
12     fmt.Println("main function")
13 }

```

Las Goroutines, *dentro del mismo programa*, pueden comunicarse entre sí mediante canales síncronos, inspirados en el paradigma Communicating Sequential Processes (CSP, de Hoare) [1]. Los canales se pueden considerar como una tubería a través de la cual se comunican las Goroutines. Desde un punto de vista semántico, los canales síncronos bloquean al emisor y al receptor hasta que ambos estén en ejecución simultánea en el canal, esto es, si un proceso escribe en el canal, este se quedará bloqueado hasta que el proceso receptor lea del canal. Y al revés, si un proceso receptor intenta leer de un canal antes de que el proceso productor escriba, también se quedará bloqueado.

En el Fragmento de Código Gorutina simple en orden ⁶, puede verse cómo pueden combinarse las Goroutines y los canales. El programa principal crea un canal de booleanos en la línea 12 y lanza la Goroutine hello en la línea 13, pasándole el canal como argumento. A partir de ahí, la Goroutine y el programa principal se sincronizarán a través del canal. Una vez que hello termine su ejecución, en la línea 9 le enviará el booleano true al programa principal. Nótese que el flujo de ejecución del programa principal estaba bloqueado, esperando, en la línea 14, hasta que la Goroutine hello escribiera en el canal.

```

1 package main // Código Gorutina simple en orden
2
3 import (
4     'fmt'
5 )
6
7 func hello(done chan bool) {
8     fmt.Println('Hello world goroutine')
9     done <- true
10 }
11 func main() {
12     done := make(chan bool)
13     go hello(done)
14     <-done
15     fmt.Println("main function")
16 }

```

⁶Disponible y ejecutable en <https://play.golang.org/p/I8goKv6ZMF>

Práctica 1: Conceptos y mecanismos básicos

Debido a la semántica de los canales síncronos, mediante la cual bloquean tanto al lector como al escritor, para aquellos escenarios en los que una Goroutine tiene que utilizar varios canales simultáneamente es necesaria una estructura de control que permita bloquearse en todos ellos y despertarse en cuanto uno esté disponible. La estructura de control `Select` ⁷ permite a una Goroutine esperar en múltiples canales simultáneamente, tanto en escritura como en lectura. Podemos tener cualquier número de declaraciones de casos dentro de `Select`, siempre habrá un canal en la guarda de cada declaración. Por tanto, al invocar `Select`, si ningún canal está listo, `Select` bloqueará al proceso invocante.

En cuanto haya una declaración activa (un canal listo para leer o escribir) se desbloqueará. Si hubiera varios activos a la vez, el sistema elige uno aleatoriamente.

En el Fragmento de código de la función `handleMessages`, hay un ejemplo ilustrativo de cómo se pueden combinar las Goroutines y los canales síncronos para solucionar el problema de la sección crítica. El fragmento corresponde a un servidor de un chat grupal centralizado que podéis encontrar en esta dirección ⁸.

```

1 func handleMessages(msgchan <-chan string,
2     addchan <-chan Client,
3     rmchan <-chan Client) { // Función handleMessages
4
5     clients := make(map[net.Conn]chan<- string)
6     for {
7         select {
8             case msg := <-msgchan:
9                 log.Printf("New message: %s", msg)
10                for _, ch := range clients {
11                    go func(mch chan<- string) {
12                        mch <- "\033[1;33;40m" + msg + "\033[m"
13                    }(ch)
14                }
15            case client := <-addchan:
16                log.Printf("New client: %v\n", client.conn)
17                clients[client.conn] = client.ch
18            case client := <-rmchan:
19                log.Printf("Client disconnects: %v\n", client.conn)
20                delete(clients, client.conn)
21        }
22    }
23 }
```

La Goroutine `handleMessages` gestiona la lista de personas que están activas en el chat durante la ejecución (variable `clients`), que es una tabla hash, una estructura de datos que almacena los participantes en el chat. A la Goroutine le llegan peticiones para añadir clientes al chat, borrar clientes del chat o para enviar mensajes a todos los participantes del chat. Notar que, aunque las peticiones pueden llegar a la vez, se atienden de una en una. Por último y no menos importante, múltiples Goroutines pueden leer

⁷<https://golangr.com/select/>

⁸https://github.com/akrennmair/telnet-chat/blob/master/03_chat/chat.go

Práctica 1: Conceptos y mecanismos básicos

simultáneamente del mismo canal hasta que este se cierre y múltiples Goroutines pueden escribir simultáneamente en el mismo canal hasta que este se cierre. Estas dos formas de utilizarlos se denominan Fan-out y Fan-in respectivamente. Tenéis más información al respecto en este link ⁹, estos patrones pueden ser muy útiles para construir la arquitectura máster-worker de esta práctica.

6. Programación Distribuida en Go

6.1. El formato de los datos

La comunicación entre procesos en un sistema distribuido requiere del intercambio de mensajes y estos a su vez se estructuran en datos. Estos datos deben serializarse para su transporte a través de la red de comunicación. En esta sección vamos a reseñar brevemente cuál es la funcionalidad que proporciona el lenguaje Go para este aspecto de la comunicación.

Si bien cuando la comunicación entre procesos se realiza a través de TCP o UDP, los procesos se intercambian información en forma de secuencias de bytes, los lenguajes de programación utilizan frecuentemente estructuras de datos para codificar y solucionar problemas computacionales. Cuando las estructuras de datos son multidimensionales o cuando muestran un tamaño variable, la transmisión de la información a través de la red se convierte en un reto, que aumenta a medida que aumenta el número de procesos del sistema y su heterogeneidad (sistema operativo, arquitectura hardware, etc.).

Una opción para poder realizar la comunicación, quizá la más básica, es que tanto el emisor como el receptor de los mensajes acuerden exactamente cómo se va a efectuar la serialización de las estructuras de datos en los mensajes (por serialización se entiende, cómo transformar una estructura de datos a una secuencia de bytes). Ningún tipo de descripción acerca de la codificación (marshalling en inglés) se incluye en el mensaje. A esta aproximación se le denomina implícita u opaca, porque tanto emisor como receptor tienen que saber cómo decodificar (unmarshalling en inglés) la información recibida a partir de una secuencia de bytes. Esta opción puede utilizarse en Go.

Frente a la estrategia implícita u opaca, se establece una aproximación que describe la estructura de los datos y se incorpora en los mensajes (metadatos); de manera que el proceso decodificador (unmarshaller) utilizará los metadatos del mensaje para extraer los datos de la secuencia binaria. Esta es la aproximación que se puede utilizar en Go; además Go utiliza su propio estándar, denominado Gob ¹⁰, que incorpora información sobre cómo se ha llevado a cabo la codificación (metadatos) dentro del mensaje, esto persigue un marshalling y unmarshalling (codificación y decodificación) más robustos. En la especificación del paquete Gob podéis encontrar ejemplos de utilización. Puede ser muy útil a la hora de realizar los ejercicios de esta práctica, de lo contrario el paso de mensajes puede convertirse en una labor muy tediosa.

⁹<https://go.dev/blog/pipelines>

¹⁰<https://pkg.go.dev/encoding/gob>

6.2. Generación de Timestamps para Medir el Tiempo de Ejecución

Los sistemas operativos proporcionan cierto soporte para poder medir el tiempo de ejecución de las operaciones, así como para cualquier otra funcionalidad relacionada con aspectos temporales. Apoyado en esas llamadas al sistema, Go proporciona el paquete `time`. En esta práctica puede ser de utilidad medir el tiempo de ejecución total de ciertas operaciones. En el fragmento de código del cliente de esta práctica, que se proporciona, se puede ver un ejemplo de utilización.

```
1 // fragmento de código correspondiente al cliente de esta práctica
2     start := time.Now()
3     id := 1
4     err = encoder.Encode(int64(40000))
5     if err != nil {
6         log.Fatal("encode error:", err)
7     }
8
9     var pisequence string
10    err = decoder.Decode(&pisequence)
11    end := time.Now()
12    fmt.Println(id, "\t", end.Sub(start))
```

La variable `start` almacena el tiempo en el instante de ejecución en que se ejecuta la instrucción `Now()` del paquete `time`. A continuación, se ejecutan una secuencia de operaciones, después se vuelve a medir el tiempo y se almacena en la variable `end`. Finalmente, se imprime por pantalla el tiempo transcurrido desde `start` a `end`, que se obtiene de restar `end - start`, operación proporcionada por el paquete `time`. De esta forma se puede medir el tiempo de ejecución de una secuencia de operaciones y esta técnica se conoce como habitualmente instrumentación del código y en este caso es intrusivo.

Referencias

- [1] Charles Antony Richard Hoare. Communicating sequential processes. *Communications of the ACM*, 21(8):666–677, 1978.